Rapport de TP Cybersécurité

Communication client/serveur avec les sockets Python et le multithreading

Module : Cybersécurité
Réalisé par : [Ton nom et prénom]
Filière / Groupe : [Ta filière]
Encadrant : [Nom du professeur]
Date : 5 novembre 2026

---

1. Introduction

Dans ce TP, on a joué un scénario simple : une machine « attaquant » qui fait tourner un serveur, et des machines « victimes » qui s'y connectent comme clients. Le but n'était pas de pirater quoi que ce soit, mais de comprendre comment deux programmes se parlent à travers un réseau. C'est la base de presque tout ce qu'on utilise sur Internet : messageries, sites web, jeux en ligne.

En cybersécurité, on ne peut pas protéger ce qu'on ne comprend pas. Savoir comment une connexion s'ouvre, comment les messages circulent et où ils peuvent être lus par quelqu'un d'autre, c'est la première étape avant de parler de défense.

Ce rapport explique ce qu'on a fait, comment on l'a fait, les problèmes qu'on a rencontrés, et ce qu'on en retient du point de vue sécurité.

2. Quelques définitions simples

Terme	Explication simple
Réseau	Des appareils reliés entre eux (par Wi-Fi ou câble) pour échanger des données.
Adresse IP	Le « numéro de téléphone » d'une machine sur le réseau. Exemple : `10.37.133.193`.

Terme	Explication simple
Port	Une « porte » numérotée sur une machine. L'IP dit quelle machine, le port dit quel programme. Nous avons utilisé les ports 8008 puis 5000.
Socket	Le point de branchement d'un programme au réseau. C'est comme une prise : on la crée, puis on s'en sert pour envoyer et recevoir.
TCP	Un protocole (une règle de communication) fiable : il vérifie que les données arrivent entières et dans le bon ordre.
Serveur	Le programme qui attend qu'on vienne le contacter.
Client	Le programme qui contacte le serveur.
Thread (processus léger)	Une petite tâche qui s'exécute en même temps que d'autres dans le même programme.
Exception	Une erreur qui arrive pendant l'exécution (par exemple un client qui coupe brutalement).
Pare-feu	Un filtre qui autorise ou bloque les connexions entrantes et sortantes.

3. Environnement du TP

- Une machine physique pour le **serveur** (le rôle de l'attaquant).
- Trois machines physiques pour les **clients** (les victimes).
- Toutes les machines sous Windows, avec Python installé.
- Le code a été écrit dans Visual Studio Code ou dans le Bloc-notes, et lancé depuis l'invite de commandes (cmd) ou le terminal de VS Code.
- Un réseau Wi-Fi commun à toutes les machines.

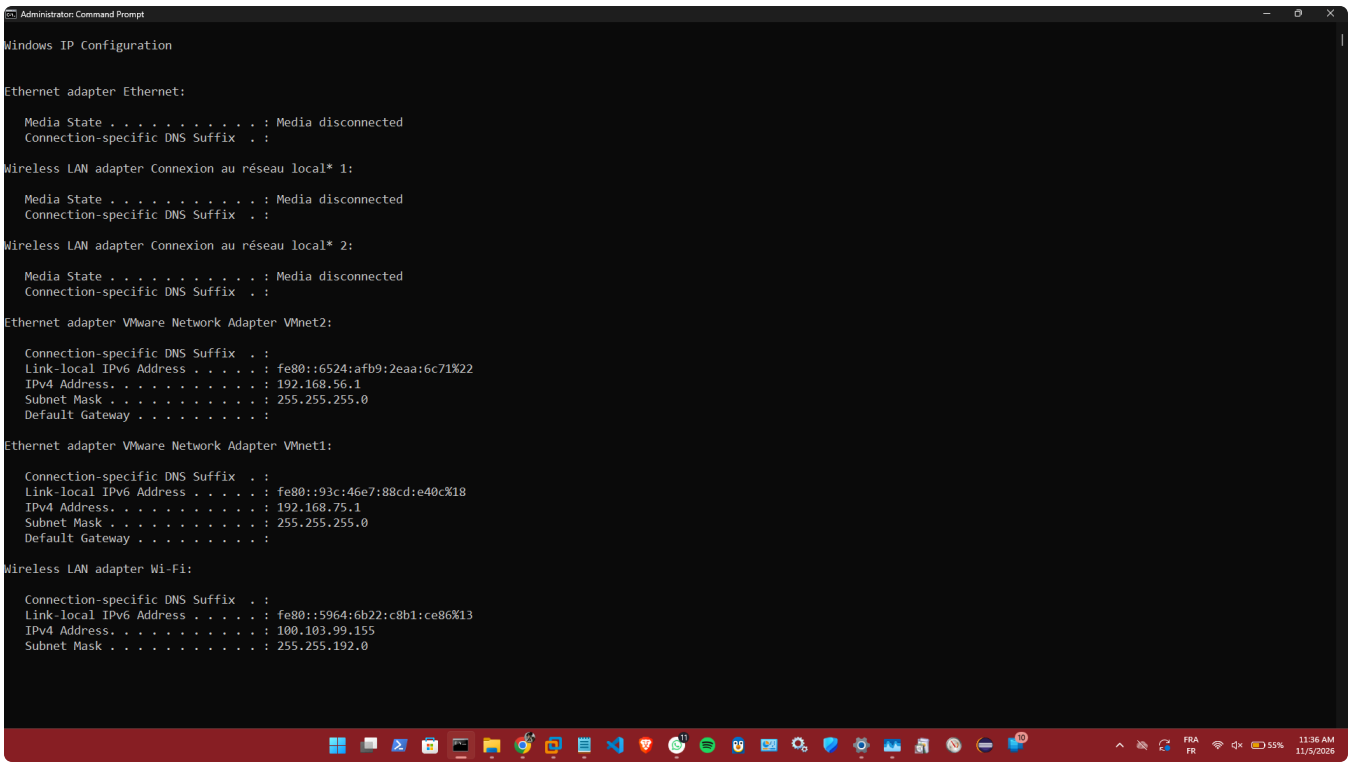


Figure 1 : la commande ipconfig donne les adresses IP de la machine. On y voit la carte Wi-Fi et les cartes virtuelles VMware.

4. Première étape : un serveur et un client simples

On est parti du code montré en cours : un serveur qui écoute, accepte une connexion, envoie un message (« Message from hacker ») puis ferme. Le client se connecte, lit le message et l'affiche.

Les étapes du serveur sont toujours les mêmes :

1. `socket.socket(...)` : créer la socket.
2. `bind(...)` : choisir l'adresse et le port sur lesquels écouter.
3. `listen(5)` : se mettre en écoute (jusqu'à 5 clients en file d'attente).
4. `accept()` : attendre qu'un client se connecte. Cette fonction **bloque** le programme tant que personne ne vient.
5. `send()` / `recv()` : envoyer et recevoir des messages.
6. `close()` : fermer la connexion.

Côté client, c'est plus court : créer la socket, `connect()` vers l'IP et le port du serveur, puis `recv()` et `send()`.

Notre premier test entre deux machines a bien marché. Le serveur (IP `100.103.99.155`) affichait `Connexion de ('100.103.99.45', 60461)` puis `Message sent`, et le client recevait le message.

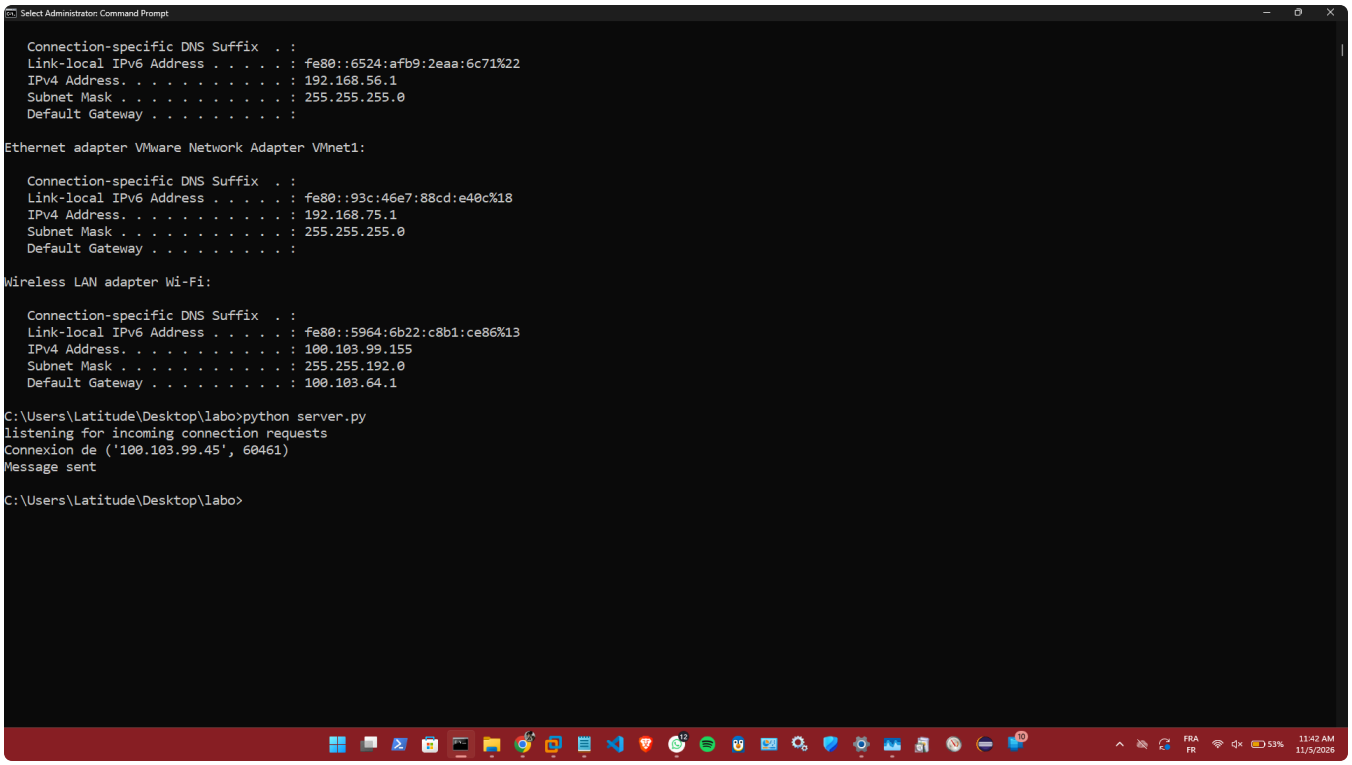


Figure 2 : premier test, le serveur reçoit la connexion du client puis envoie son message.

5. Deuxième étape : échanger des messages dans les deux sens

Le professeur a demandé que le client et le serveur puissent à la fois envoyer et recevoir. Le problème, c'est que `input()` (pour taper un message) et `recv()` (pour attendre un message) bloquent tous les deux. Si on les met dans la même boucle, on ne peut faire qu'une chose à la fois.

La solution est d'utiliser un **thread** : un thread s'occupe de la réception en arrière-plan, pendant que le programme principal s'occupe de l'envoi.

On a testé cette version entre deux machines. Le serveur affichait par exemple `Client : bonjour`, et les messages se croisaient dans les deux sens. Quand le client tapait `quitter`, le serveur affichait « Le client a quitté la conversation ».

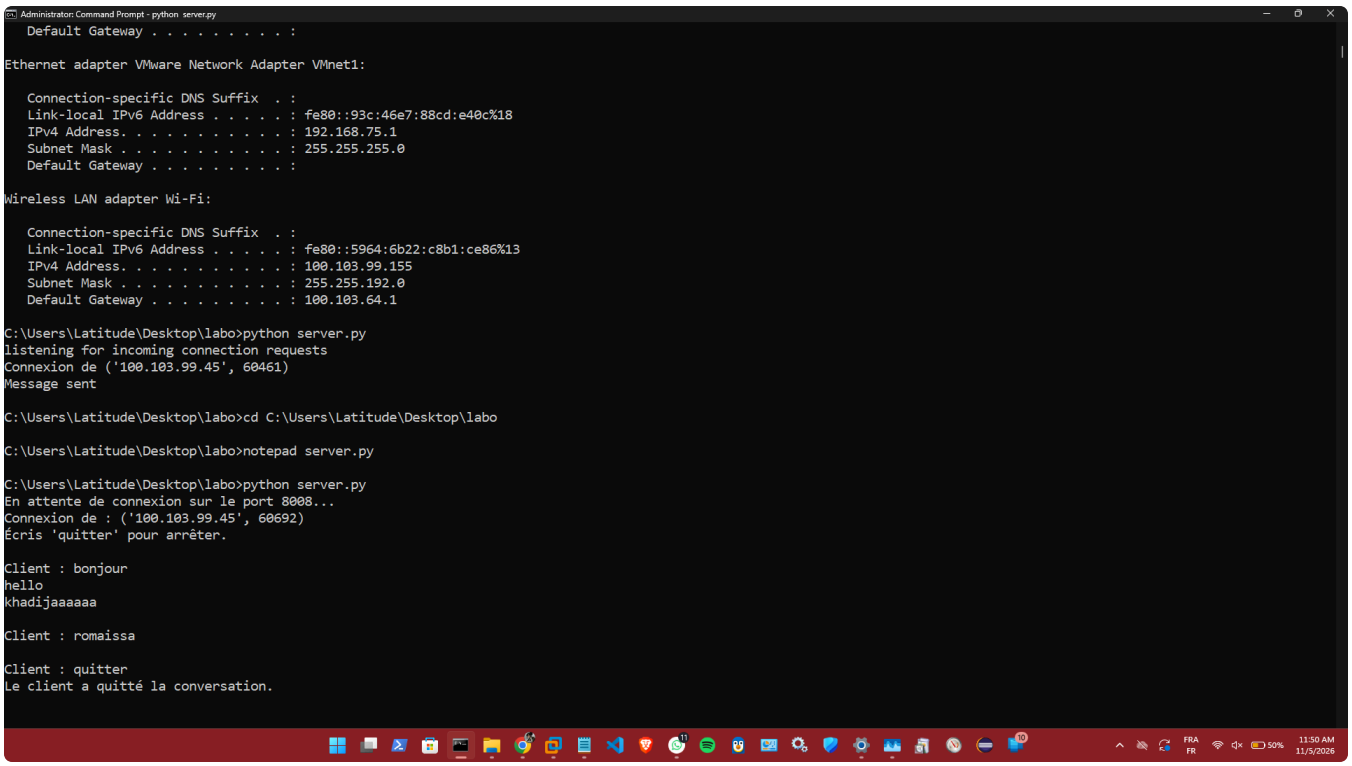


Figure 3 : côté serveur, les messages du client s'affichent et le serveur peut répondre. Le client termine avec « quitter ».

6. Troisième étape : plusieurs clients en même temps

Le problème

Avec un seul `accept()`, le serveur ne peut parler qu'à un client à la fois. Les autres attendent dans la file d'attente de `listen()`. Si la conversation avec le premier client dure longtemps, les autres finissent par recevoir une erreur de délai dépassé (timeout). Ce n'est pas acceptable pour un vrai serveur.

La solution

Le thread principal ne fait qu'une seule chose : il tourne en boucle sur `accept()`. Dès qu'un client arrive, il crée un **nouveau thread** pour s'occuper uniquement de ce client, puis retourne attendre le suivant. Comme ça, tous les clients sont servis en parallèle.

La partie importante du code est celle-ci :

```
while True:
    conn, addr = server.accept()
    t = threading.Thread(target=traiter_client, args=(conn, addr))
    t.start()
```

Deux détails qu'on a dû bien comprendre :

- `target=traiter_client` donne **le nom** de la fonction, sans parenthèses. C'est le thread qui l'appellera lui-même (c'est ce qu'on appelle une fonction de rappel, ou *callback*). Si on avait écrit `traiter_client(conn, addr)`, la fonction se serait exécutée tout de suite dans le programme principal et le serveur aurait été bloqué à nouveau.
- Écrire `def traiter_client(...)` ne l'exécute pas. Python enregistre seulement la fonction. Elle ne tourne que quand un thread l'appelle.

La gestion des exceptions

Chaque thread client utilise un `try / except`. L'idée n'est pas de faire disparaître les erreurs, mais d'**empêcher qu'une erreur chez un client ne coupe tout le serveur**. Si un client ferme sa fenêtre brutalement, l'erreur est attrapée, la connexion de ce client est fermée, et les autres continuent comme si de rien n'était.

Identifier les clients

Pour savoir facilement qui envoie quoi, le serveur numérote les clients (Client 1, Client 2, etc.) et affiche leur adresse IP à la connexion. Une variante consiste à demander au client d'envoyer son prénom en premier message, pour l'afficher à la place de l'adresse IP.

7. Résultats du test final

Pour le test final, le serveur tournait sur une machine et plusieurs clients se sont connectés depuis d'autres machines du réseau (adresses en `10.37.133.x`). La console du serveur montre :

- les connexions successives : `[Client 2 connecté : ('10.37.133.34', 63396)]`, `[Client 3 connecté : ('10.37.133.193', 50793)]`, `[Client 4 connecté : ('10.37.133.34', 63448)]`, puis plus tard un Client 5 ;
- les messages reçus avec le numéro du client : `[Client 3] hello i'm romaissa` ;
- les réponses envoyées par le serveur : `Réponse à client 3 : hey` ;

- les déconnexions : `[Client 2 déconnecté : …]`, `[Client 1 déconnecté : …]`.

Ce qu'il faut retenir, c'est que plusieurs clients ont discuté **en même temps** avec le serveur, et que le départ d'un client n'a pas gêné les autres. Le multithreading fait bien son travail.

Sur la machine d'un client, on voit la conversation :

```
Connecté au serveur. Tapez 'quit' pour sortir.
Vous : hello i'm romaissa
hey
Vous : hello i'm romaissa
salam romaissaa
Vous : quit
Déconnexion.
```

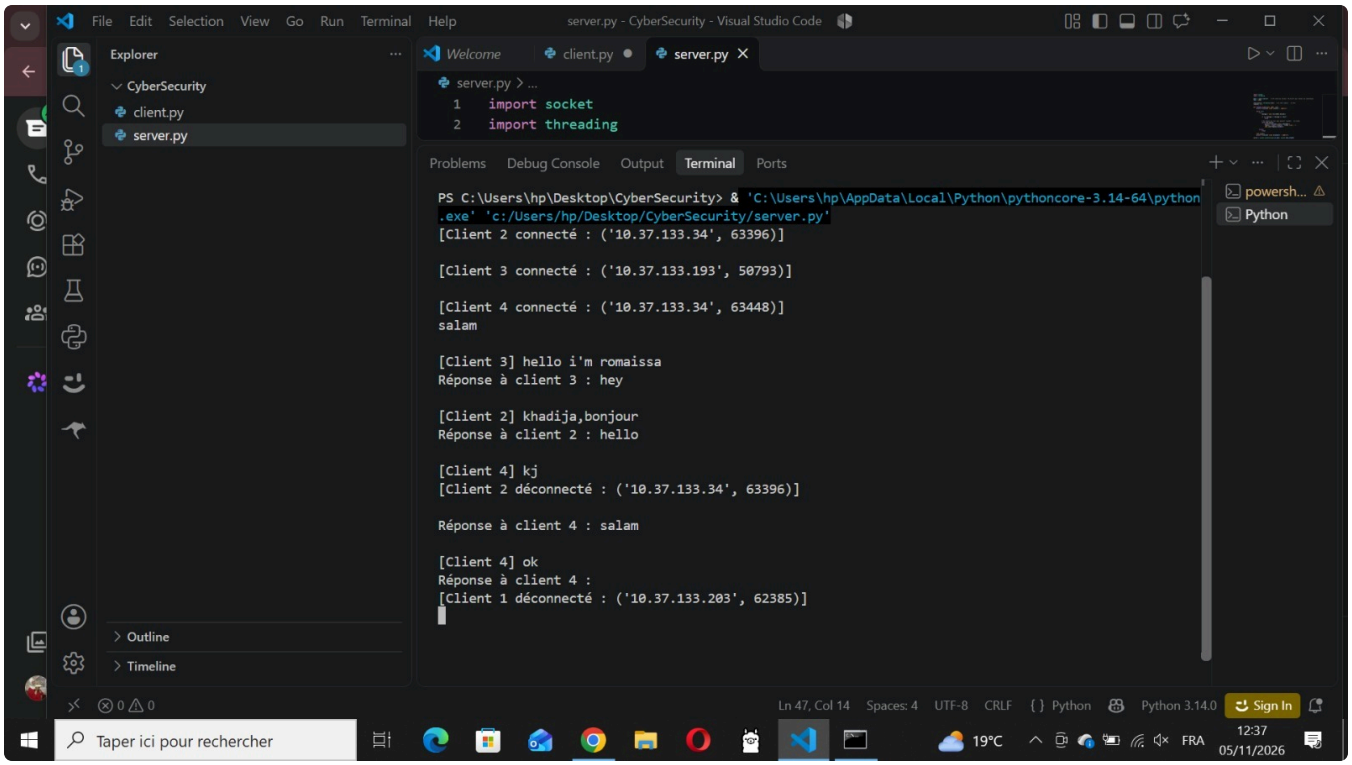


Figure 4 : la console du serveur montre les clients 2, 3 et 4 connectés en même temps, leurs messages et les réponses.

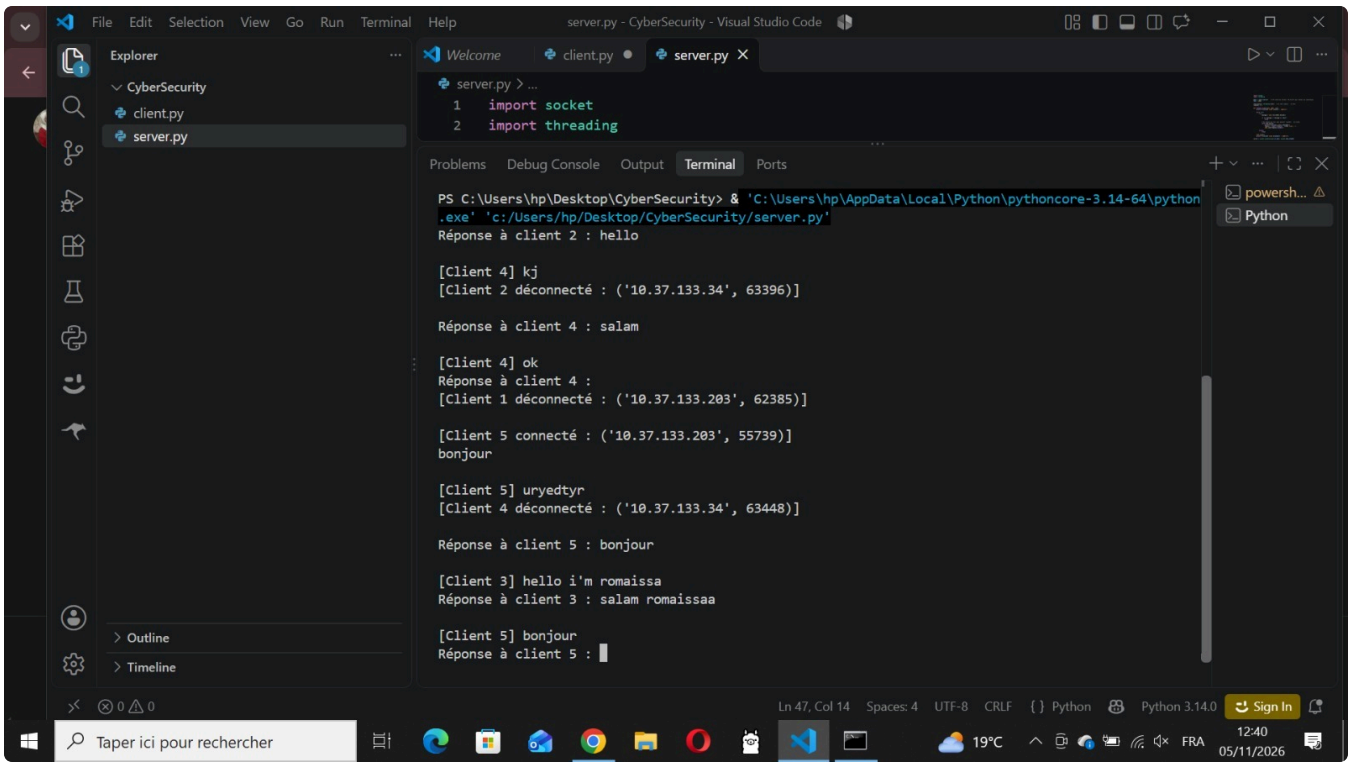


Figure 5 : la suite du test, avec les déconnexions et l'arrivée d'un nouveau client (Client 5).

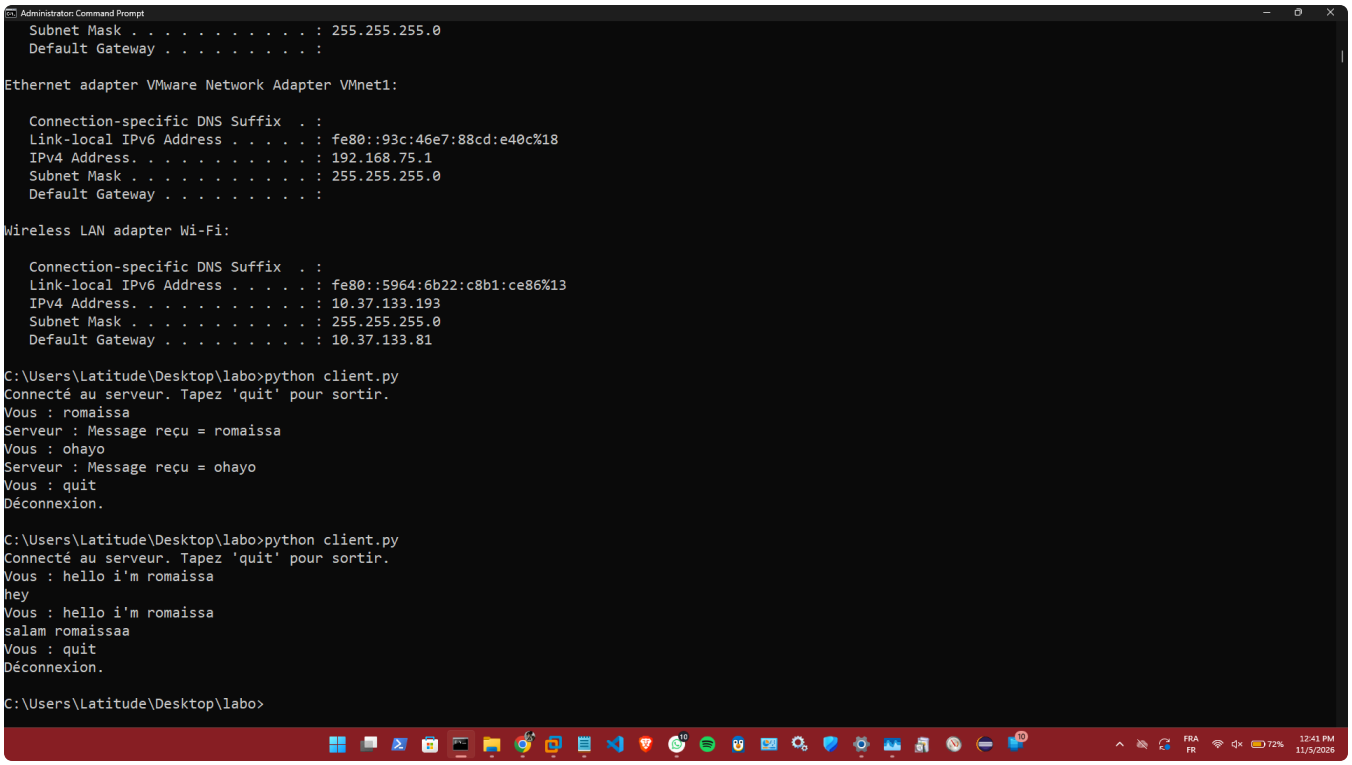


Figure 6 : côté client, les messages envoyés (« Vous : … ») et les réponses reçues, jusqu'à la déconnexion avec « quit ».

8. Les problèmes rencontrés

On a perdu pas mal de temps sur des erreurs bêtes, mais qui nous ont appris des choses :

Problème	Cause	Solution
`'conn.send' is not recognized as an internal or external command`	On avait collé le code Python directement dans cmd, qui ne comprend que les commandes Windows.	Écrire le code dans un fichier `.py` (Bloc-notes ou VS Code) puis le lancer avec `python server.py`.
Le serveur refusait de démarrer avec l'adresse de la slide	L'IP `192.168.74.128` appartenait à la machine du professeur, pas à la nôtre. `bind()` ne peut utiliser qu'une adresse de **sa propre** machine.	Utiliser `0.0.0.0` (toutes les cartes réseau) côté serveur.
Le client ne pouvait pas se connecter depuis un autre PC	Avec `127.0.0.1`, le serveur n'accepte que les connexions de sa propre machine.	`0.0.0.0` côté serveur, et l'IP réelle du serveur côté client (trouvée avec `ipconfig`).
Connexion impossible entre machines	Pare-feu Windows, ou réseau qui isole les appareils les uns des autres (réseau en `100.x.x.x`).	Autoriser Python dans le pare-feu, tester avec `ping`, et passer sur un réseau où les machines se voient (`10.37.133.x`).
L'IP du serveur changeait	L'adresse dépend du réseau Wi-Fi utilisé.	Refaire `ipconfig` à chaque changement de réseau et mettre à jour le client.

Un point utile : la commande `ipconfig` nous a servi plusieurs fois, d'abord pour trouver l'IP du serveur, ensuite pour comprendre pourquoi les machines ne se voyaient pas. Elle affiche aussi les cartes virtuelles de VMware (`192.168.75.1`, `192.168.56.1`), qu'il ne faut pas confondre avec la carte Wi-Fi réelle.

9. Analyse du point de vue sécurité

Ce TP montre surtout ce qui **manque** dans une communication basique.

Les messages circulent en clair. Notre code envoie le texte tel quel avec `send()`. Quelqu'un qui est sur le même réseau et qui capture le trafic (avec un outil comme Wireshark, par exemple) pourrait lire tous les messages. Nous n'avons pas fait cette capture dans ce TP, mais c'est une suite logique pour montrer le problème.

Aucune authentification. N'importe qui qui connaît l'IP et le port peut se connecter au serveur. Le serveur ne vérifie pas qui est en face.

Un port ouvert est une surface d'attaque. Plus un serveur écoute sur `0.0.0.0` avec un port ouvert, plus il est exposé. C'est pour ça que le pare-feu existe : il décide qui a le droit d'entrer.

Pas de limite de connexions. Notre serveur crée un thread par client sans limite. Si quelqu'un ouvrait des centaines de connexions d'un coup, la machine pourrait saturer. C'est le principe d'un déni de service.

Les mesures de protection possibles :

- chiffrer les échanges avec TLS (le « S » de HTTPS) ;
- demander une authentification (mot de passe, certificat) ;
- limiter le nombre de connexions simultanées ;
- n'ouvrir que les ports nécessaires et configurer le pare-feu ;
- journaliser les connexions pour pouvoir détecter les comportements suspects.

Tout ce TP a été réalisé sur un réseau de travail, entre machines participantes et avec l'accord de l'encadrant. Utiliser ce genre de code contre des machines qui ne nous appartiennent pas serait illégal.

10. Conclusion

Ce TP nous a permis de comprendre concrètement comment fonctionne une communication client/serveur : une socket, une adresse, un port, et un échange de messages en TCP. On a aussi vu pourquoi un serveur réel doit être **concurrent** : avec un thread par client, plusieurs personnes peuvent être servies en même temps, et la gestion des exceptions évite qu'une erreur arrête tout le monde.

Le plus formateur a été de rencontrer les vrais problèmes de réseau (mauvaise adresse, pare-feu, réseau qui isole les appareils), qu'on ne voit pas quand on teste sur une seule machine.

Sur le plan de la sécurité, on retient que notre petit programme fonctionne, mais qu'il n'est pas protégé : tout passe en clair et n'importe qui peut se connecter. Dans un vrai projet, il faudrait ajouter du chiffrement et de l'authentification.

Annexe : code utilisé

server.py (version multi-clients)

```
import socket
import threading

def traiter_client(conn, addr):
    print("Client connecté :", addr)

    while True:
        try:
            message = conn.recv(1024).decode()

            if not message or message == "quit":
                break

            print("Message reçu :", message)
            reponse = "Serveur : Message reçu = " + message
            conn.send(reponse.encode())

        except OSError:
            break

    conn.close()
    print("Client déconnecté :", addr)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 5000))
server.listen(5)
print("Serveur démarré...")

while True:
    conn, addr = server.accept()
    t = threading.Thread(target=traiter_client, args=(conn, addr))
    t.start()
```

client.py

```
import socket
import threading

IP_SERVEUR = "IP_DU_SERVEUR"   # trouvée avec ipconfig sur le PC serveur
PORT = 5000

def recevoir(sock):
    while True:
        try:
            data = sock.recv(1024).decode()
        except OSError:
            break
        if not data:
            print("\nServeur déconnecté.")
            break
        print("\n" + data)

nom = input("Ton nom : ")

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((IP_SERVEUR, PORT))
client.send(nom.encode())
print("Connecté. Écris 'quit' pour quitter.")

threading.Thread(target=recevoir, args=(client,), daemon=True).start()

while True:
    message = input()
    try:
        client.send(message.encode())
    except OSError:
        break
    if message == "quit":
        break

client.close()
```